

ET UNIFIED OPERATIONS BRAIN

COMPLETE SYSTEM DOCUMENTATION & STEP-BY-STEP OPERATIONS MANUAL

Document Version:	1.0.0	Target System OS:	Windows / Linux / macOS
System Name:	ET Unified Operations Brain	Architecture:	Hybrid Microservices (4 Backends + 1 Frontend)

ET UNIFIED OPERATIONS BRAIN: COMPLETE PROJECT DOCUMENTATION

Document Version: 1.0.0

1. EXECUTIVE OVERVIEW & SYSTEM PURPOSE

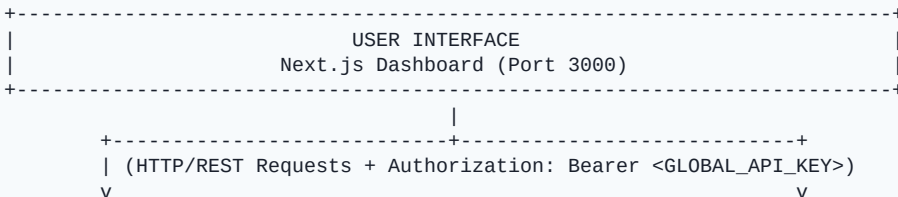
The ET Unified Operations Brain is an enterprise-grade, multi-agent AI system designed for industrial refinery operations, heavy equipment maintenance, regulatory compliance, and grounded operational intelligence.

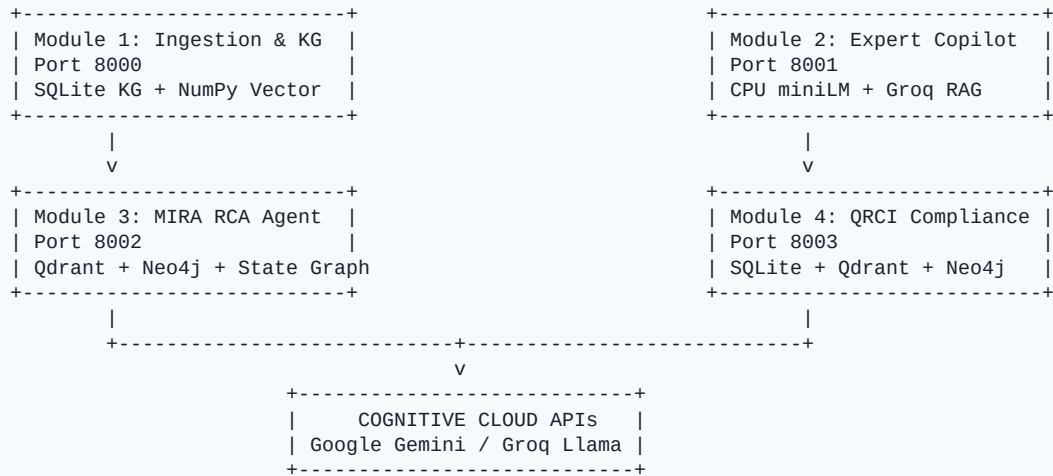
The platform unifies state-of-the-art Large Language Models (Google Gemini & Groq Llama 3.3/3.1), Knowledge Graphs (SQLite Node-Edge & Neo4j), dense vector search (Qdrant & local CPU/NumPy embeddings), and hybrid multi-paradigm database layers.

It provides real-time document ingestion, automated entity/relationship extraction, grounded retrieval-augmented generation (RAG) with strict citation guarantees, stateful agentic 5-Why root cause analysis (RCA), real-time sensor quality deviation detection, and automated regulatory compliance gap analysis.

2. SYSTEM ARCHITECTURE & DATA FLOW

The platform is structured into four specialized Python FastAPI backend services and one Next.js modern web dashboard frontend.





Data Flow Workflow:

- 1. Ingestion: Raw files (PDFs, spreadsheets, images) or Gmail operational emails are sent to Module 1. Text is parsed, OCR'd (if scanned), chunked, embedded (Gemini Embeddings), and structured into entity-relationship graphs.
- 2. Grounded Copilot: Module 2 ingests document contexts, runs hybrid search (TF-IDF + CPU Vector Similarity + Graph Boost), dynamically routes query complexity between Llama-3.1-8b and Llama-3.3-70b, and verifies citations.
- 3. Maintenance RCA (MIRA): Module 3 monitors equipment telemetry, executes stateful 5-Why hypothesis graphs, runs multi-constraint resource solvers (budget/labor/safety), and identifies root causes.
- 4. Regulatory Compliance (QRCI): Module 4 runs continuous compliance scans, evaluates deterministic rules (e.g. emission limits) and qualitative checks, compiles audit evidence packages, and assesses regulatory amendment impacts.
- 5. Unified UI: Next.js frontend interacts with all 4 endpoints independently via the shared `GLOBAL\_API\_KEY` header.

3. COMPLETE CODEBASE DIRECTORY STRUCTURE

```
ETHackathon-Human-Intelligence/
■■■■ .env # Central backend environment configuration
■■■■ .gitignore # Git exclusion rules
■■■■ ClearData.py # Standalone system data purge & cache reset utility
■■■■ EnvRequirements.txt # Unified python package dependencies list
■■■■ LoginDetails.txt # System credentials & sample account reference
■■■■ README.md # Root documentation & setup guide
■■■■ run_servers.py # Multi-process orchestrator (with logging & auto-restart)
■■■■ start_all.py # Lightweight gateway script (Docker/cloud hosting)
■■
■■■■ frontend/ # Next.js 16 Web Dashboard User Interface
■■ ■■■■ .env.local # Frontend environment & backend API URLs configuration
■■ ■■■■ package.json # Node dependencies (React 19, Motion, Lucide, Supabase)
■■ ■■■■ next.config.ts # Next.js workspace configurations
■■ ■■■■ tsconfig.json # TypeScript compiler options
■■ ■■■■ public/ # Static web assets & SVG illustrations
```

■ ■■■■ src/	# Source application code
■ ■■■■ app/	# Next.js App Router pages & layout components
■ ■■■■ components/	# UI design components (cards, graphs, sidebar, modals)
■ ■■■■ lib/	# API client wrappers & Supabase client setup
■	
■■■■ ML MODELS/	# Four Backend Microservices
■■■■ Md 1/	# Module 1: Ingestion & Knowledge Graph Subsystem
■ ■■■■ README.md	# Module 1 documentation
■ ■■■■ backend/	# Core Python application
■ ■■■■ config.py	# Paths, API keys, and operational settings
■ ■■■■ database.py	# SQLite schema design (Nodes, Edges, Documents)
■ ■■■■ parser_service.py	# Universal document parsers (PDF, CSV/XLSX, OCR)
■ ■■■■ vector_service.py	# NumPy cosine similarity & chunking engine
■ ■■■■ gemini_service.py	# Multi-model LLM extraction & fallback routines
■ ■■■■ gmail_service.py	# OAuth 2.0 Gmail & mock email sync
■ ■■■■ main.py	# FastAPI backend server (Port 8000)
■ ■■■■ cli.py	# Terminal interactive console dashboard
■ ■■■■ verify_backend.py	# Automated integration test runner
■ ■■■■ reset_data.py	# Local module reset utility
■ ■■■■ uploads/	# Ingested file repository
■ ■■■■ outputs/	# Generated queries, mindmaps, and flowcharts
■	
■■■■ Md 2/	# Module 2: Groq-Powered RAG & Expert Copilot
■ ■■■■ README.md	# Module 2 documentation
■ ■■■■ main.py	# FastAPI backend server (Port 8001)
■ ■■■■ rag_core.py	# Hybrid search, dynamic router, & citation gate
■ ■■■■ visualization.py	# Mindmap & Mermaid flowchart generator
■ ■■■■ evaluation.py	# Benchmark test runner (DocVQA, FUNSD, OSHA)
■ ■■■■ requirements.txt	# Module 2 dependencies
■ ■■■■ data/	# Evaluation benchmark datasets
■	
■■■■ Md 3/	# Module 3: Maintenance Intelligence & RCA (MIRA)
■ ■■■■ Mod 3/	# Module 3 documentation
■ ■■■■ README.md	# FastAPI backend server (Port 8002)
■ ■■■■ main.py	# LangGraph stateful 5-Why RCA & solver graphs
■ ■■■■ agents.py	# Relational schema (assets, WOs, sensors, RCA)
■ ■■■■ data_store.py	# Swappable Qdrant vector database layer
■ ■■■■ vector_db.py	# Neo4j Graph DB adapter & local JSON mock
■ ■■■■ graph_db.py	# LLM client wrapper
■ ■■■■ gemini_client.py	# Module 3 configuration settings
■ ■■■■ config.py	# Offline mock graph fallback schema
■ ■■■■ mock_graph.json	# Detailed MIRA architecture spec
■ ■■■■ architecture.md	# Execution workflow documentation
■ ■■■■ workflow.md	
■	
■■■■ Md 4/	# Module 4: Quality & Regulatory Compliance (QRCI)
■ ■■■■ Mod 4/	# Module 4 documentation
■ ■■■■ README.md	# FastAPI backend server (Port 8003)
■ ■■■■ main.py	# Compliance scan, gap analysis, & deviation subgraphs
■ ■■■■ agents.py	# Relational tables, rules engine, & seed data
■ ■■■■ data_store.py	# Qdrant regulations semantic search layer
■ ■■■■ vector_db.py	# Neo4j Graph DB adapter & local mock fallback
■ ■■■■ graph_db.py	# LLM client wrapper
■ ■■■■ gemini_client.py	# Module 4 configuration settings
■ ■■■■ config.py	# Offline mock graph schema
■ ■■■■ mock_graph_compliance.json	# Synthetic datasets for seeding plant data
■ ■■■■ Datasets/	

4. DETAILED MODULE BREAKDOWN & CAPABILITIES

MODULE 1: Universal Document Ingestion & Knowledge Graph Subsystem

- Purpose: Converts unstructured enterprise files into structured vector embeddings

and explicit relational knowledge graph nodes/edges.

- Technical Mechanics:
- Multimodal Parsing: Uses PyMuPDF for native text. Automatically switches to Gemini Multimodal Vision OCR if text density is under 100 characters/page.
- Tabular Intelligence: Uses Pandas to translate CSV/XLSX into Markdown tables or column summary schema previews depending on size.
- Overlapping Chunker: Generates 800-character text chunks with 150-character overlaps to retain cross-boundary context.
- Vector Store: Computes 768d float arrays using Gemini Embeddings and performs cosine similarity in RAM using pure NumPy arrays (no heavy external vector daemon).
- Multi-Model LLM Fallback Chain: Primary extraction runs on Groq Llama-3.3-70B. If rate-limited, falls back across Gemini 2.0/2.5/3.5 models. If offline, runs deterministic Regex rule-based extractors.
- SQLite Graph Database: Stores normalized nodes, edges, document join tables, Q&A history, and rendered mindmaps.

MODULE 2: Groq-Powered RAG & Expert Knowledge Copilot

- Purpose: Grounded decision co-pilot delivering high-speed, cited responses without reliance on external commercial embedding APIs.
- Technical Mechanics:
- CPU Embeddings: Employs `all-MiniLM-L6-v2` locally on CPU (384 dimensions) with L2-normalized outputs.
- Hybrid Retrieval: Blends vector similarity scores, TF-IDF exact token matches, and SQLite graph node degree boosting.
- Dynamic LLM Router: Simple queries (<15 words, single entity) are sent to `llama-3.1-8b-instant`. Multi-entity or complex reasoning queries are routed to `llama-3.3-70b-versatile`.
- Citation Validation Gate: Post-processor that checks every bracketed source citation `[Doc\_ID]`. If a citation cannot be validated against database records, the system automatically triggers a strict self-correction re-generation loop.
- Evaluation Harness: Includes benchmark suites for DocVQA, FUNSD forms, and OSHA safety incident narratives.

MODULE 3: Maintenance Intelligence & RCA Agent (MIRA)

- Purpose: Operational maintenance assistant providing 5-Why root cause analysis (RCA), fleet asset health monitoring, and resource-constrained job scheduling.
- Technical Mechanics:
- Stateful Agentic Graph: Models a multi-turn LangGraph architecture (`RCASState`) with supervisor routing, evidence verification, and human-in-the-loop interrupters.
- Multi-Constraint Solver: Uses an optimization algorithm to prioritize work orders against labor availability, total budget, safety criticality, and equipment

parent-child dependencies.

- Dual Search Layer: Uses Qdrant vector database for maintenance manual RAG and Neo4j Graph DB for topology traversal (with fallback to local SQLite/JSON).
- Relational Database: Manages `assets`, `work\_orders`, `sensor\_tags`, `inspection\_findings`, `rca\_sessions`, `rca\_nodes`, and `recommendations`.

MODULE 4: Quality & Regulatory Compliance Intelligence (QRCI)

- Purpose: Automated compliance monitoring engine for safety, environmental, and pressure vessel regulations (Factories Act, PESO, OISD, IBR).
- Technical Mechanics:
 - 3-Tier Database Engine: Relational SQLite (records & rules), Qdrant Vector DB (regulatory clauses), and Neo4j Graph DB (`Requirement APPLIES\_TO Asset`).
 - Continuous Compliance Scans: Evaluates numeric limits (e.g. pH boundaries 5.5-9.0, SO2 limits ≤ 50) using deterministic python rules, and qualitative safety requirements using LLM reasoning.
 - Real-Time Quality Deviations: Ingests sensor streams and inspection reports, flagging critical/major/minor deviations and synthesizing remedial actions.
 - Regulatory Amendment Sandbox: Performs diff-analyses on amended regulations, tracing asset graphs to report affected machinery and required procedural changes.
 - Synthetic Data Ingestion: Auto-seeds 30 machinery units, 120 work orders, 60 inspection reports, and 25 safety incident logs on database initialization.

FRONTEND DASHBOARD (Next.js 16 + React 19)

- Purpose: Executive & operational control interface for viewing plant status, uploading files, executing copilot queries, viewing mindmaps, and managing RCA.
- Tech Stack: Next.js 16, React 19, Tailwind CSS v4, Motion (framer-motion successor), Lucide icons, Lottie animations, Supabase Auth.
- Multi-Backend Interconnect: Makes REST requests directly to ports 8000, 8001, 8002, and 8003 using the `Authorization: Bearer <GLOBAL\_API\_KEY>` header.

5. SYSTEM ADVANTAGES & TECHNICAL SUPERIORITY

1. Zero-Downtime Multi-Model Fallbacks:

The ingestion & RAG engines never fail due to API exhaustion. If Groq limits are exceeded, requests shift to Gemini; if Gemini is offline, the system falls back to CPU-based local embeddings and deterministic regex extractors.

2. Hybrid Tri-Database Architecture:

Combining Relational SQLite (structured records), Vector DB (semantic context), and Graph DB (topology & relationship links) eliminates the blind spots of single-database

RAG solutions.

3. Deterministic Safety Enforcement:

Critical regulatory thresholds (e.g., pH levels, pressure bounds, gas limits) are verified by Python code rules rather than hallucination-prone LLMs.

4. Self-Correcting Citation Integrity:

Every answer generated by the co-pilot passes through a citation gate. Hallucinated sources trigger a retry mechanism before the answer is delivered to the user.

5. Lightweight CPU Operation:

Modules 1 & 2 run locally without requiring expensive GPU infrastructure, utilizing NumPy vectors and CPU-optimized `all-MiniLM-L6-v2` embeddings.

6. Autonomous Process Management:

`run\_servers.py` continuously monitors all 4 microservice processes, logging stdout to disk and auto-restarting crashed instances automatically.

6. KEY UNIQUENESS & INNOVATION

- Graph-Boosted Vector Retrieval: Graph degree connectivity directly boosts document relevance rankings in RAG search queries.
- Dynamic Query Routing: Automatically conserves token costs by assigning simple queries to lightweight 8B models and reserving 70B models for complex queries.
- Stateful 5-Why RCA Workbench: Guides maintenance engineers step-by-step through root-cause evidence verification with checkpointed states.
- Regulatory Amendment Diff Engine: Predicts plant-wide equipment impacts when government regulations update.
- Single-Command System Purge: `ClearData.py` completely resets SQLite databases, vector caches, uploads, outputs, and Next.js build artifacts in seconds.

7. FEATURE USAGE & FUNCTIONALITY MANUAL

Feature 1: Document Upload & Knowledge Graph Construction

- Endpoint: POST <http://localhost:8000/documents/upload>
- CLI Usage: Run `python cli.py` in Module 1 backend -> Select Option 1.
- Functionality: Ingests PDF/Image/CSV files. Automatically parses pages, extracts entities/relations, indexes vectors, and updates graph nodes.

Feature 2: Grounded Copilot Q&A with Citation Validation

- Endpoint: POST <http://localhost:8001/query>

- CLI Usage: Run ``python cli_rag.py`` in Module 2 -> Select Option 1.
- Functionality: Submit natural language operational queries. Receive answers with exact bracketed source references ``[doc_id]`` and confidence ratings.

Feature 3: Mindmap & Flowchart Generation

- Endpoint: POST `http://localhost:8000/visualize/mindmap` or `/visualize/flowchart`
- CLI Usage: Run ``python cli.py`` in Module 1 -> Select Mindmap/Flowchart options.
- Functionality: Converts document extractions into hierarchical mindmap JSON structures or valid Mermaid.js sequential workflow diagrams.

Feature 4: Stateful 5-Why RCA Workbench (MIRA)

- Endpoint: POST `http://localhost:8002/rca/start`
- CLI Usage: Run ``python cli_agent.py`` in Module 3 -> Select RCA Workbench.
- Functionality: Initiate an RCA session for a failed asset (e.g. Pump P-204). Iteratively generates 5-Why hypothesis branches backed by sensor findings.

Feature 5: Maintenance Resource & Constraint Solver

- Endpoint: POST `http://localhost:8002/solver/optimize`
- CLI Usage: Run ``python cli_agent.py`` in Module 3 -> Select Constraint Solver.
- Functionality: Input total available maintenance budget and labor hours. Generates an optimized list of high-priority work orders to execute first.

Feature 6: Continuous Compliance Gap Scan (QRCI)

- Endpoint: POST `http://localhost:8003/compliance/scan`
- CLI Usage: Run ``python cli_agent.py`` in Module 4 -> Select Compliance Scan.
- Functionality: Audits equipment against safety standards (OISD/PESO/IBR). Generates compliance gap reports and evidence packages.

Feature 7: Real-Time Quality Deviation Alerting

- Endpoint: POST `http://localhost:8003/deviations/check`
- CLI Usage: Run ``python cli_agent.py`` in Module 4 -> Select Deviation Checker.
- Functionality: Monitors incoming sensor telemetry or inspection logs, flagging critical parameter breaches and outputting remedial steps.

8. STEP-BY-STEP HOW-TO-OPERATE MANUAL

STEP 1: Prerequisites Verification

Ensure the following tools are installed on your host system:

- Python: Version 3.10 or 3.11 (``python --version``)
- Node.js: Version 18.x or later (``node -v``)

- npm: Version 9.x or later (`npm -v`)
- Git: Installed and configured (`git --version`)
- Docker Desktop: Optional, required only for containerized deployment.

STEP 2: Environment Configuration

1. Root Backend Environment File (`.env`):

Create `.env` in the root repository directory (`ETHackathon-Human-Intelligence/.env`):

```
GEMINI_API_KEY=your_gemini_api_key_here
GROQ_API_KEY=your_groq_api_key_here
GLOBAL_API_KEY=et_brain_secure_key_2026_xyz
NEO4J_URI=
NEO4J_USER=neo4j
NEO4J_PASSWORD=password
```

Note: If Neo4j is not installed, leave `NEO4J\_URI=` blank. System automatically uses local SQLite/JSON mock graph fallbacks.

2. Frontend Environment File (`frontend/.env.local`):

Create `.env.local` in `frontend/.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-supabase-project-id.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key-here
MODULE1_API_URL=http://localhost:8000
MODULE2_API_URL=http://localhost:8001
MODULE3_API_URL=http://localhost:8002
MODULE4_API_URL=http://localhost:8003
```

STEP 3: Launching the Backend Microservices

Option A: Local Python Orchestrator (Recommended for Development)

1. Open terminal at repository root directory.
2. Create and activate a Python virtual environment:

```
python -m venv venv
# On Windows:
venv\Scripts\activate
# On macOS/Linux:
source venv/bin/activate
```

3. Install requirements (Note: paths with spaces must be enclosed in quotes):

```
pip install fastapi uvicorn
pip install -r "ML MODELS/Md 1/backend/requirements.txt"
pip install -r "ML MODELS/Md 2/requirements.txt"
pip install -r "ML MODELS/Md 3/Mod 3/requirements.txt"
pip install -r "ML MODELS/Md 4/Mod 4/requirements.txt"
```

4. Launch all 4 servers concurrently:

```
python run_servers.py
```

- Servers start on ports 8000, 8001, 8002, 8003.
- Logs are directed to `logs/module1.log` through `logs/module4.log`.
- To stop all backends, press `Ctrl + C`.

Option B: Docker Container Deployment (Production Mode)

1. Ensure Docker Desktop is running.
2. From the root repository directory, execute:

```
docker compose up --build
```

3. All 4 microservices will compile and run inside Docker containers.

STEP 4: Launching the Frontend Web Dashboard

1. Open a new terminal window and navigate to `frontend/`:

```
cd frontend
```

2. Install Node dependencies:

```
npm install
```

3. Start the Next.js development server:

```
npm run dev
```

4. Open your browser and navigate to:

<http://localhost:3000>

STEP 5: Running Interactive Developer CLI Dashboards

Each backend module includes an interactive terminal interface:

- Module 1 CLI:

```
cd "ML MODELS/Md 1/backend"
```

```
python cli.py
```

- Module 2 Copilot CLI:

```
cd "ML MODELS/Md 2"
```

```
python cli_rag.py (or run.bat)
```

- Module 3 MIRA CLI:

```
cd "ML MODELS/Md 3/Mod 3"
```

```
python cli_agent.py
```

- Module 4 Compliance CLI:

```
cd "ML MODELS/Md 4/Mod 4"
```

```
python cli_agent.py
```

STEP 6: Executing Automated Tests & Benchmark Suites

- Test Module 1 Backend:

```
cd "ML MODELS/Md 1/backend" && python verify_backend.py
```

- Test Module 2 RAG & Evaluation Benchmark:

```
cd "ML MODELS/Md 2" && python evaluation.py
```

- Test Module 3 MIRA Integration Suite:

```
cd "ML MODELS/Md 3/Mod 3" && python test_agent.py
```

- Test Module 4 Compliance Suite:

```
cd "ML MODELS/Md 4/Mod 4" && python test_agent.py
```

STEP 7: System Data Reset & Purge Operations

To hard-reset the system back to a clean state:

1. Stop all backend servers (`run\_servers.py` or Docker) and frontend dev server.
2. From the root directory, run:

```
python ClearData.py
```

3. This utility empties all SQLite tables, deletes uploaded files, purges generated mindmaps/flowcharts, removes JSON/vector caches, and clears the frontend ``.next`` build cache.

9. TROUBLESHOOTING & FREQUENTLY ASKED QUESTIONS (FAQ)

Q1: API Requests fail with "401 Unauthorized"

A1: Check that ``GLOBAL_API_KEY`` is defined in root ``.env`` and matches between the backend environment and frontend requests (``Bearer <GLOBAL_API_KEY>``).

Q2: System hangs or times out on startup waiting for Neo4j

A2: If Neo4j is offline or not installed, leave ``NEO4J_URI=`` blank in your root ``.env``. ``run_servers.py`` automatically sets ``NEO4J_URI=""`` to force instant local mock graph fallbacks.

Q3: "File path not found" or syntax errors when running pip install on Windows

A3: Folder names under ``ML MODELS`` contain spaces. Always wrap file paths in double quotes, **for example:** ``pip install -r "ML MODELS/Md 2/requirements.txt"``.

Q4: Port collision errors (e.g. "Address already in use: 8000")

A4: Ensure no lingering Python or Uvicorn processes are running. Terminate them via:

Windows PowerShell: ``Stop-Process -Name python -Force``

Linux/macOS: ``kill -f uvicorn``

Q5: Module 2 complains about missing vector embedding files

A5: Module 2 downloads ``all-MiniLM-L6-v2`` via HuggingFace transformers on first run.

Ensure active internet access during initial launch.

END OF DOCUMENTATION MANUAL
